Autonomous Drone Racing

Session 4: Code, Tools, Hardware

Marcel Rath

Chair of Safety, Performance and Reliability for Learning Systems
School of Computation, Information and Technology

LEARNING SYSTEMS &
ROBOTICS LAB

What We Cover Today

1. Software Development* [45 min]
2. Hardware Intro [45 min]
3. Time for Questions [25 min]

*Part of the content was adopted from the tutorial, “Code repository setup: lessons learned & getting hands-on with a Python repository,” by Frederike Dumbgen and Olivier Lamarre

Evaluation Criteria: Quality of Solution

Criteria	w. t.	100% means …	0% means …
Algorithm Robustness	30%	Reliably finishes level 0 to 3 in simulation, is resilient to multiple noise settings, manages to finish the real track in the drone lab	Drone constantly crashes, can't finish level 0 in simulation
Speed/ Performance	30%	Drone crosses all gates, significant speed improvement over the baseline controller, fastest group in the race	Drone does not remain on track, same or slower speed than the baseline, last group in the race
Methodology & Design	20%	Systematic approach, good planning, algorithms are theoretically justified, students performed additional literature research on their approach	Non-systematic illogical approach, relies on trivial heuristics, only parameter tuning
Code Quality	20%	Descriptive, brief functions and classes, modular code, clear naming scheme, documentation of functions, classes and critical code pieces, PEP8 conforming code	Monolithic code, confusing variable names, no documentation or comments, code doesn’t conform to any style guides

Software Development

- Learn from examples (other open-source code)
- Write readable code (from variable names to equation structure)
  - Use a proper naming convention
  - PEP 8 Style Guide for Python Code
  - Use one statement of code per line
- Comment plentily
- Test your code frequently

Quality expected: code that can be open-sourced and used by others

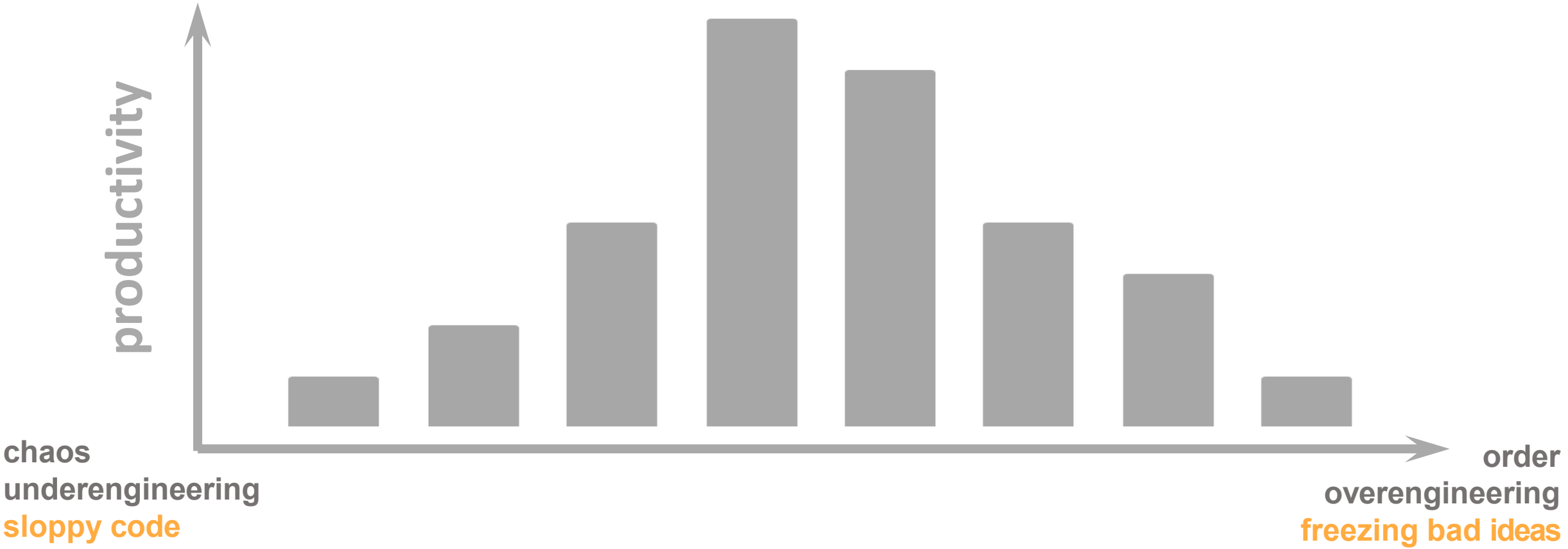
Software Development

Be aware of your goal

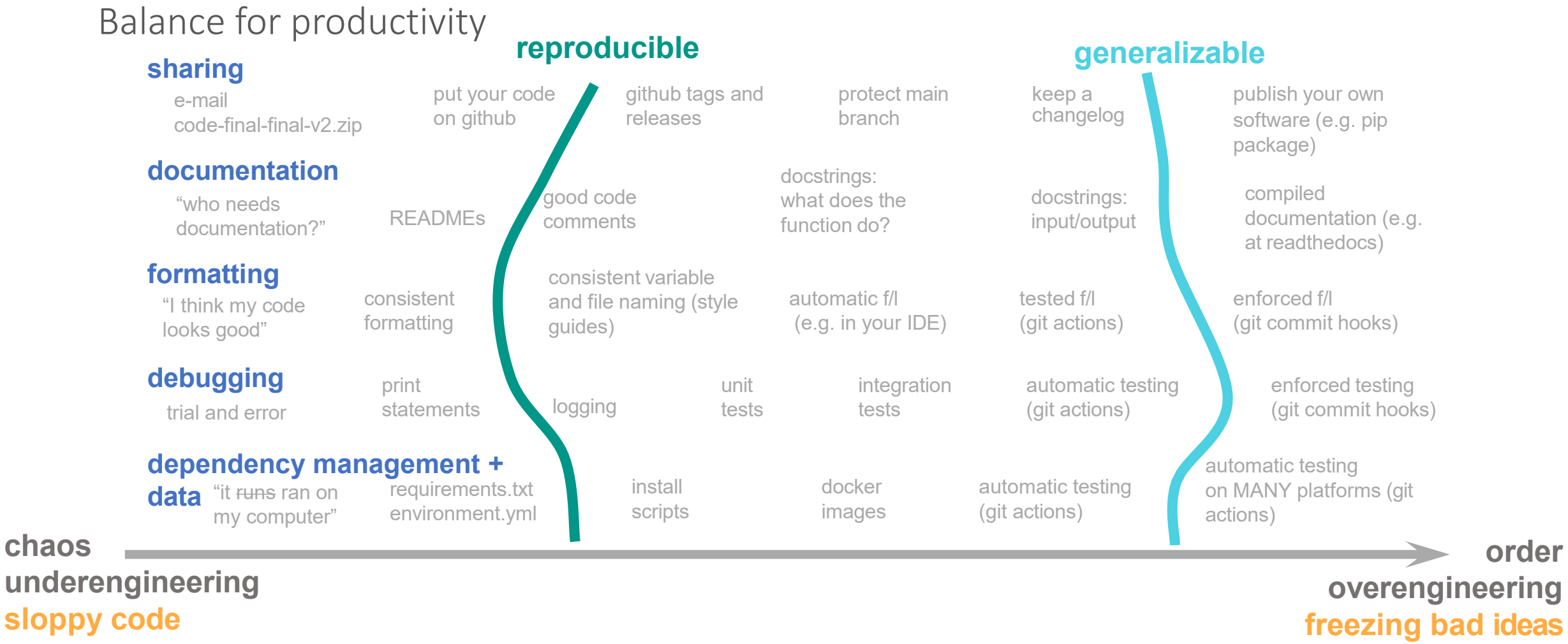
		Data or Platform	
		Same	Different
Code	Same	*Reproducible*	*Replicable*
	Different	*Robust*	*Generalizable*

Software Development

Balance for productivity

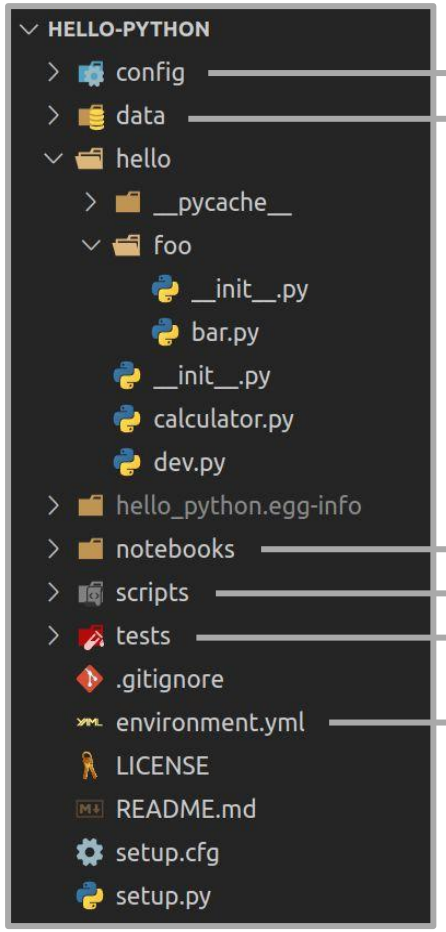


Software Development



Software Development

Keep a good structure (example https://github.com/olamarre/hello-python)



Configuration files (parameters, logging, etc.)

A directory dedicated to data products you generate (checkpoints, logs, plots, etc.). Git-ignore its contents except for small data files essential to the repository (such as a lightweight dataset for unit tests).

The main Python package of this project. By convention, it should be 1 word. Every python file inside of it is a "module" and every directory containing python modules (called a "package") must contain a `__init__.py` file.

Jupyter Notebook, Google Colab, etc.

All scripts not part of the main package. These could be high-level python files (which might import the main package, but aren't part of it, pre-processing/setup shell scripts, etc.

Unit & integration tests for functionalities defined in the main package.

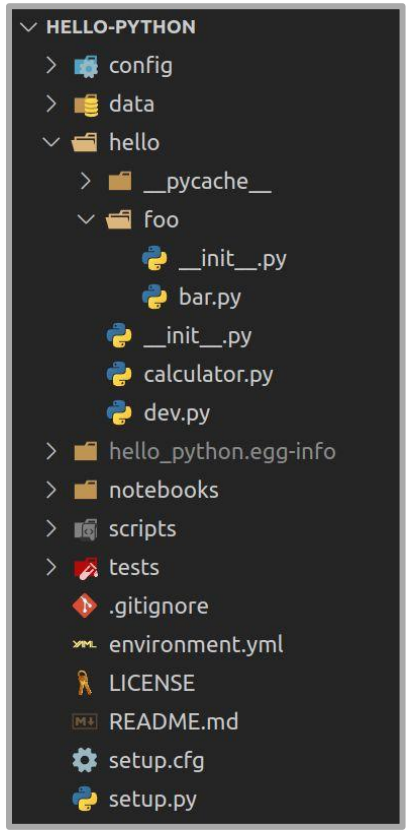
A python environment description file (for conda).
If you are only using pip, that would be requirement.txt

How others may use your code.
Repo description & basic procedures.

Necessary to install the (local) "hello" package, so you can import it in any python file in this environment.
Instead of using these two files, pyproject.toml is becoming the standard.

Software Development

Keep a good structure (example https://github.com/olamarre/hello-python)



- This is a good starting point for a robotics / data science Python project
- Depending on the application, they can be a lot more complicated and contain more directories. For example:
  - /lib: stores C extensions/libraries
  - /doc: all documentation files
  - Container (ex: Docker) configuration files
  - Continuous integration
- Many repository structure conventions exist - this is just one of them

General References

- The "basics" of CS
- Hitchhikers guide to Python
- Realpython
- Google's Python style guide
- Of course: stackoverflow
- Still highly recommended: Beyond Pep8

Software Development

sharing

e-mail
code-final-final-v2.zip

put your code on GitHub

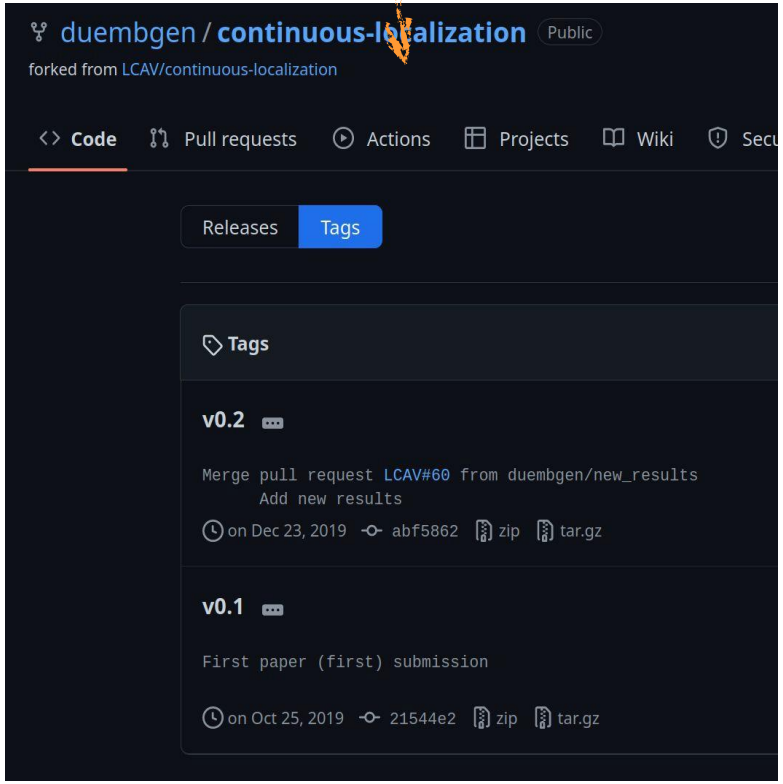
GitHub tags and releases

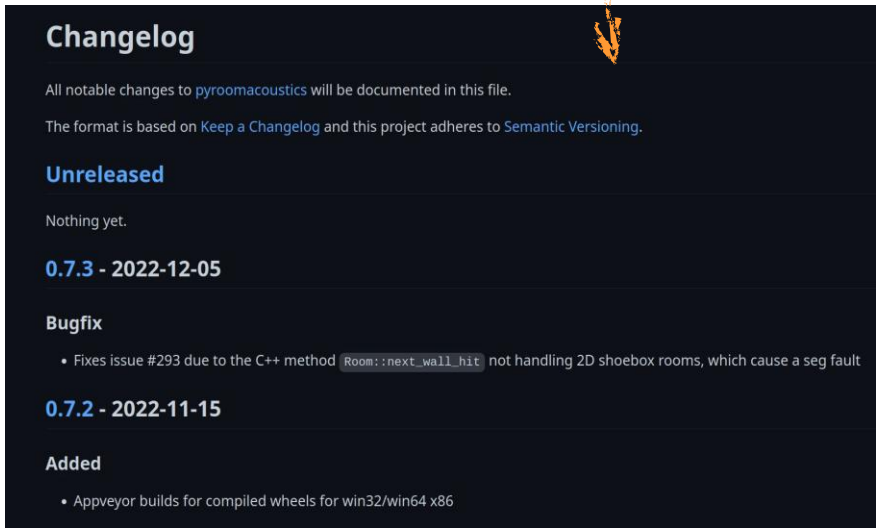
protect main branch

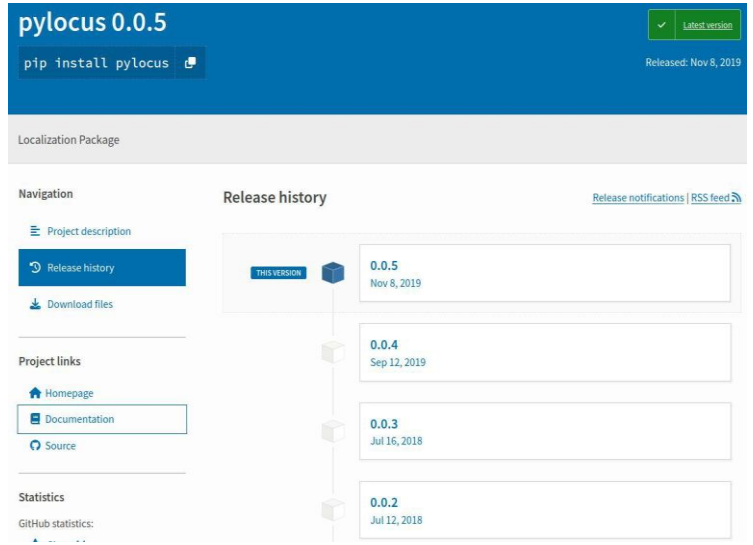
keep a changelog

publish your own software (e.g. pip package)

don't do this unless you have a good reason







Software Development

documentation

"who needs documentation?"

READMEs

good code comments

docstrings: what does the function do?

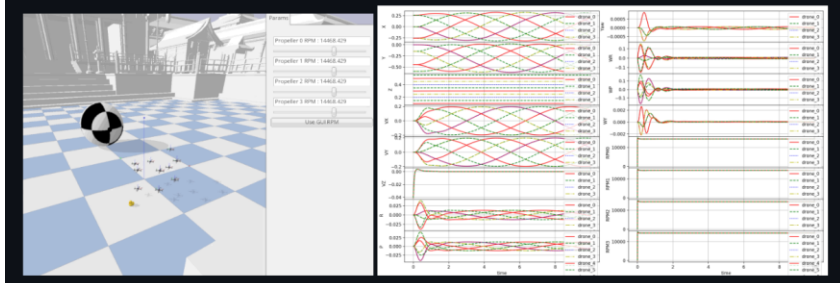
docstrings: input/output

```python
@player_pose.setter
def player_pose(self, coordinates: Tuple[float]):
    # If we write the x coordinate and the game loop updates the player's position immediately
    # after, we teleport before setting the other coordinates. In order to minimize these races
    # between coordinates, we pack xzy into a byte package and write it in one call. We can't
    # include `a` because of the memory layout, but this is less important as the orientation
    # can still be updated after a tick delay.
    buff_death = self.allow_player_death
    self.allow_player_death = False
    self.gravity = False
    base = self.mem.bases["WorldChrMan"]
    x_address = self.mem.resolve_address(self.data.address_offsets["PlayerX"], base=base)
    a_address = self.mem.resolve_address(self.data.address_offsets["PlayerA"], base=base)
    xzy = struct.pack('fff', coordinates[0], coordinates[2], coordinates[1])  # Swap y z order
    self.mem.write_bytes(x_address, xzy)
    self.mem.write_float(a_address, coordinates[3])
    self.gravity = True
    self.allow_player_death = buff_death
    self.player_hp = self.player_max_hp
```

gym-pybullet-drones

This is a minimalist refactoring of the original `gym-pybullet-drones` repository, designed for compatibility with `gymnasium`, `stable-baselines3` 2.0, and SITL `betaflight` / `crazyflie-firmware`.

> NOTE: if you prefer to access the original codebase, presented at IROS in 2021, please `git checkout [paper|master]` after cloning the repo, and refer to the corresponding `README.md`'s.



Installation

Tested on Intel x64/Ubuntu 22.04 and Apple Silicon/macOS 14.1.

```
git clone https://github.com/utiasDSL/gym-pybullet-drones.git
cd gym-pybullet-drones/

conda create -n drones python=3.10
conda activate drones

pip3 install --upgrade pip
pip3 install -e . # if needed, `sudo apt install build-essential` to install `gcc` and build `pybullet`
```

Use

PID control examples

```
cd gym_pybullet_drones/examples/
python3 pid.py # position and velocity reference
python3 pid_velocity.py # desired velocity reference
```

Software Development

documentation

"who needs documentation?"

READMEs

arguably the most important!

good code comments

docstrings: what does the function do?

docstrings: input/output

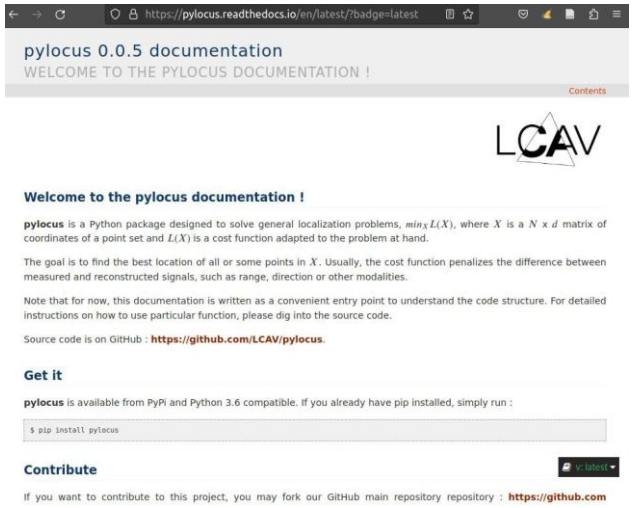
compiled documentation (e.g. at readthedocs)

Write docstrings when you need to remind yourself for the second time what a function does.
Consistent is more important than complete.

The Sphinx docstring format

In general, a typical `Sphinx` docstring has the following format:

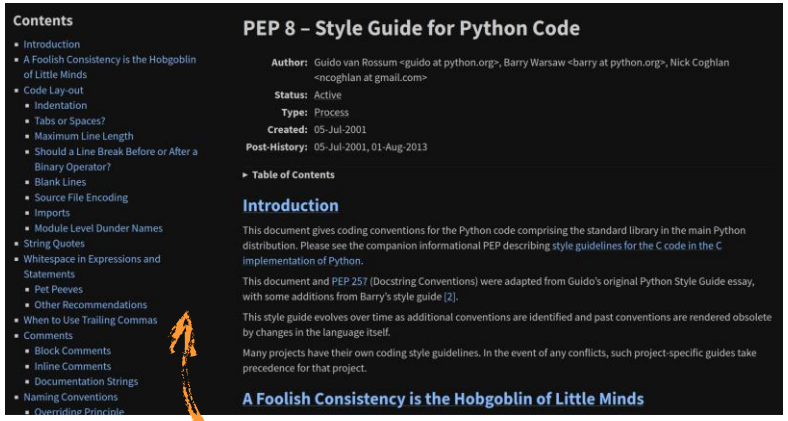
```
"""[Summary]

:param [ParamName]: [ParamDescription], defaults to [DefaultParamVal]
:type [ParamName]: [ParamType](, optional)
...
:raises [ErrorType]: [ErrorDescription]
...
:return: [ReturnDescription]
:rtype: [ReturnType]
"""
```

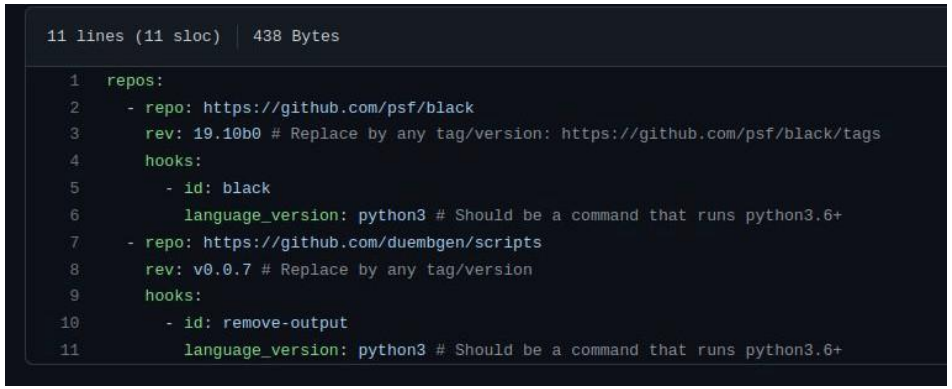


Documentation - Recommendations

- Sphinx: The go-to Python documentation package
- Napoleon: More readable docstrings (Google style)
- ReadTheDocs: Hosted, fully automated documentation

Software Development





formatting

"I think my code looks good"

consistent formatting

consistent variable and file naming (style guides)

automatic f/l (e.g. in your IDE)

zero-effort, high-reward!

tested f/l (git actions)

later more on this

enforced f/l (git commit hooks)

```
frederike@duembgen:poly_matrix$ black poly_matrix.py
reformatted poly_matrix.py

All done! ✨ 🍰 ✨
1 file reformatted.
frederike@duembgen:poly_matrix$ git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   poly_matrix.py

no changes added to commit (use "git add" and/or "git commit -a")
frederike@duembgen:poly_matrix$ git diff
diff --git a/poly_matrix.py b/poly_matrix.py
index fa6fccf..9c3213a 100644
--- a/poly_matrix.py
+++ b/poly_matrix.py
@@ -7,6 +7,7 @@ import numpy as np

 class PolyMatrix(object):
     def __init__(self):
+
         self.matrix = {}

         self.start = 0
frederike@duembgen:poly_matrix$
```

Formatting and Linting - Recommendations

Linter options

- Ruff The fastest kid on the block
- Yapf Different formatting style
- Flake8 and black: Linter and formatter. Pretty much replaced by ruff

Formatting and Linting - Ruff

Core Commands

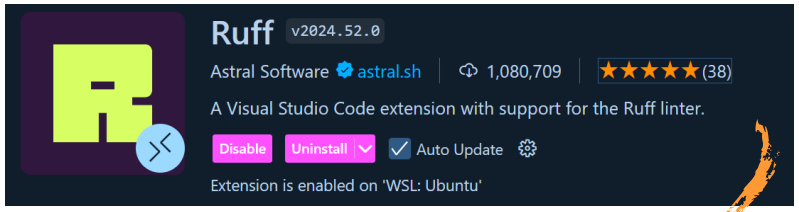
- **`ruff check`**: Scans your code for linting errors, logical bugs, and style violations.
- **`ruff check --fix`**: Scans & automatically fixes resolveable issues (e.g., removing unused imports, upgrading syntax, or sorting imports).
- **`ruff format`**: An uncompromising code formatter that ensures consistent spacing, line breaks, and indentation across your project.

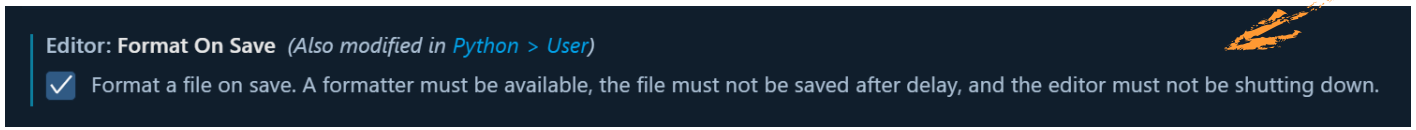
Rules

- Syntax rules are defined in **`pyproject.toml`** under **`[tool.ruff]`** session

Automated formatting (Alternative)

- Install ruff extension in VSCode
- Enable format on save





Software Development

debugging
trial and error

print statements

logging

unit tests

integration tests

automatic testing (git actions)

enforced testing (git commit hooks)

Debugging: Logging

- print() works for simple applications or quick tests
- logging is preferred with larger applications/packages

- Logging
- Logs have a level (DEBUG, INFO, WARNING, …)
- You can set the logging level to filter out messages
- The log output can be redirected (terminal, file, database, …)

Level	Numeric value
CRITICAL	50
ERROR	40
WARNING	30
INFO	20
DEBUG	10
NOTSET	0

Debugging: Logging

1. Configure how the logging messages show up in your main script
2. At the top of the module being imported, simply add: log = logging.getLogger(__name__)

main.py

```python
#!/usr/bin/env python

from pathlib import Path
import logging
import logging.config
import json

from hello.calculator import add, subtract, multiply, divide
from hello.dev import REPO_CFG

def main():

    log.info(f"In main")

    res = add(2,3)
    res = subtract(10,4)
    res = multiply(4,5)
    res = divide(5,2)


if __name__ == "__main__":

    # Create a Stream handler (for console output)
    sh = logging.StreamHandler()
    sh.setFormatter(
        logging.Formatter('%(levelname)s - %(name)s: - %(message)s'))
    sh.setLevel(logging.INFO)

    # Configure main logger, and connect it to the handlers
    logging.basicConfig(level=logging.INFO, handlers=[sh])
    log = logging.getLogger(__name__)

    main()

```

hello/calculator.py

```python
#!/usr/bin/env python

"""
    A set of calculator functions

    Author: <your name, <email>>
    Affl.: <your affiliation>
"""

import logging

log = logging.getLogger(__name__)


def add(i,j):
    """Add two numbers"""
```

A StreamHandler, by default, prints the logging messages to the console/terminal. You can specify the format in which the messages will appear.

Create the logger

Debugging: Logging

- You can use multiple handlers at once!

```python
if __name__ == "__main__":

    # Create a Stream handler (for console output)
    sh = logging.StreamHandler()
    sh.setFormatter(
        logging.Formatter('%(levelname)s - %(name)s: - %(message)s'))
    sh.setLevel(logging.INFO)

    # Also print the logging messages to a file in our repo's data directory
    log_dir = Path(REPO_CFG.data_dir, 'logs')
    log_dir.mkdir(exist_ok=True)
    log_fpath = Path(log_dir, f'{Path(__file__).stem}.log')

    fh = logging.FileHandler(filename=log_fpath, mode='w')
    fh.setFormatter(
        logging.Formatter(
            '%(asctime)s - %(levelname)s - %(name)s: - %(message)s'))
    fh.setLevel(logging.INFO)

    # Configure main logger, and connect it to the handlers
    logging.basicConfig(level=logging.INFO, handlers=[sh,fh])
    log = logging.getLogger(__name__)

    main()
```

Let's also add a FileHandler, which will save all logging outputs to a file saved in our repo's data directory.

No more crazy terminal scrolling!

```python
if __name__ == "__main__":

    # Load a logging configuration (a StreamHandler & a FileHandler)
    with open(Path(REPO_CFG.config_dir, 'logging.json')) as f:
        log_cfg = json.load(f)

    # Still need to specify the path to the log file for the FileHandler
    log_dir = Path(REPO_CFG.data_dir, 'logs')
    log_dir.mkdir(exist_ok=True)

    lfn = Path(log_dir, f'{Path(__file__).stem}.log')
    log_cfg['handlers']['file']['filename'] = lfn
    logging.config.dictConfig(log_cfg)
    log = logging.getLogger(__name__)

    main()
```

To avoid copy/pasting logging configurations in your scripts, save your logging settings in a configuration file! Look for the config/logging.json file in the sample repository.

Debugging

debugging
trial and error

print statements

logging

unit tests

integration tests

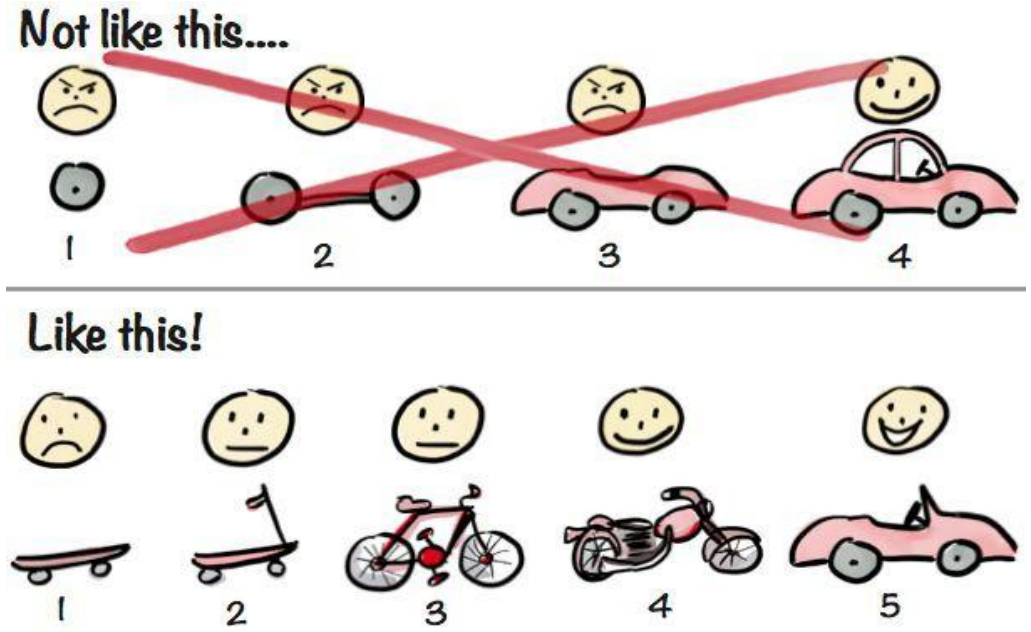
automatic testing (git actions)

enforced testing (git commit hooks)

Software Development

Debugging: Unit and Integration Tests

- unit tests: test each small part of your code
- integration tests: test full pipelines (potentially including datasets)

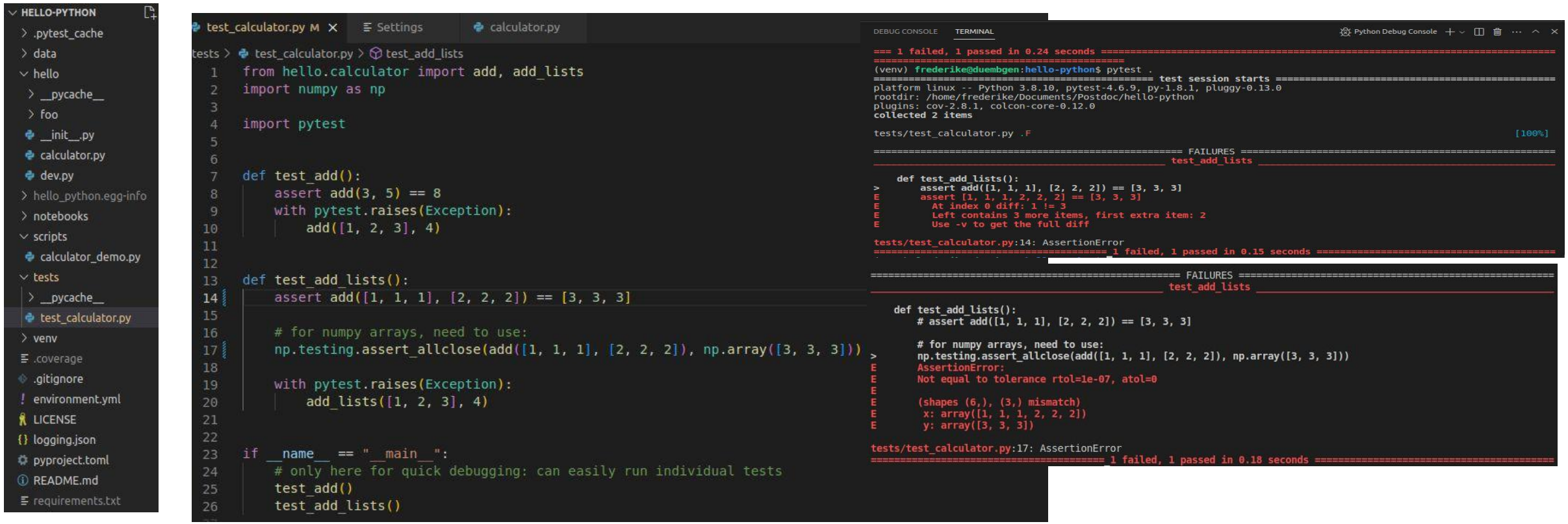


Drawing by Henrik Kniberg

Software Development

Debugging: Unit and Integration Tests

- unit tests: test each small part of your code
- integration tests: test full pipelines (potentially including datasets)



Debugging: Unit and Integration Tests

- Why this is useful:
  - Structure your workflow with test-driven development (“where do I even start?”)
  - You probably already write those in one way or another! (“wait, does this function work?”)
  - Peace of mind when reusing old code (“can I still use this code?”)

- Some example unit tests (optimization/state-estimation inspired)
  - Test gradients with finite differences
  - Assert that the gradient at your local solution is zero
  - Assert ground truth solution when measurements are noiseless

Software Development

Dependency Management and Data

dependency management + data

"it ~~runs~~ ran on my computer"

requirements.txt
environment.yml

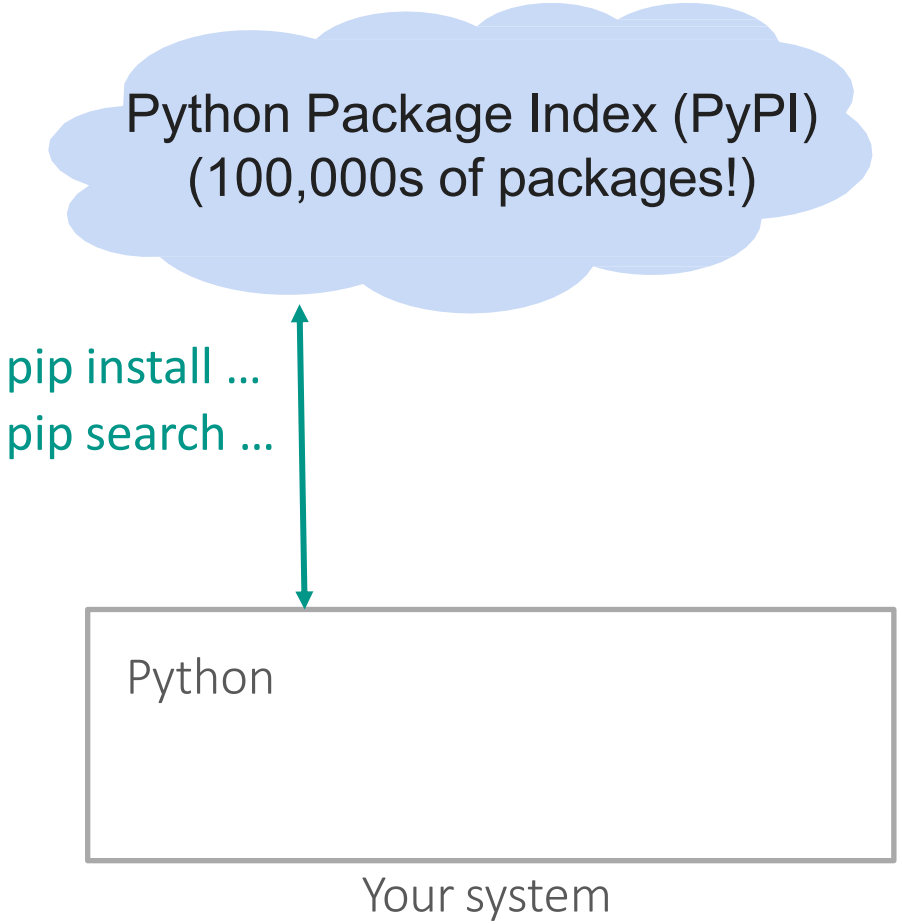
install scripts

docker images

automatic testing (git actions)

automatic testing on MANY platforms (git actions)

Dependency Management: pip vs conda



pip is Python's main package manager.

You can install non-standard packages from PyPI…

$ pip install numpy

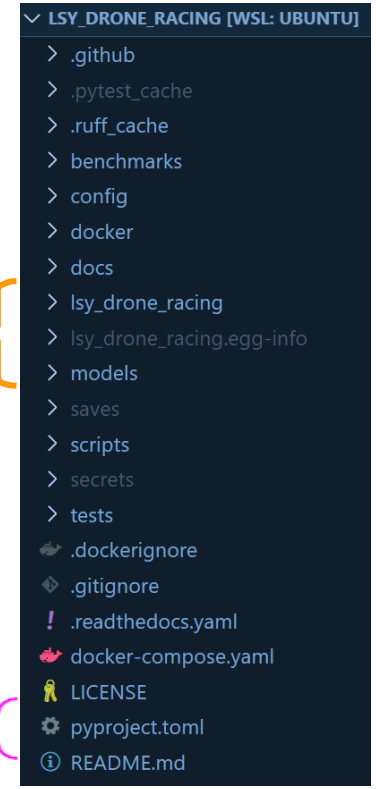
… as well as your own local package(s):

$ cd lsy_drone_racing

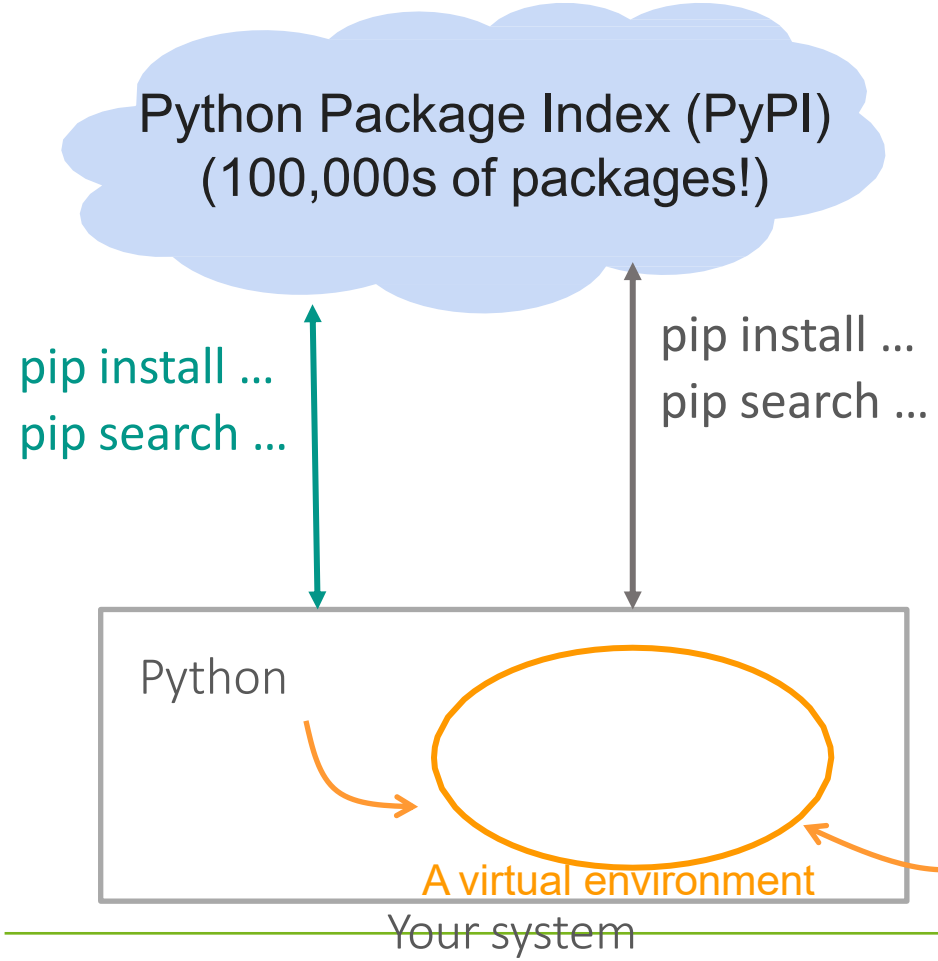
$ pip install -e .

Local installs use the project file *pyproject.toml* to make the lsy_drone_racing package importable into your python scripts

-e option: "edit" mode, lets you modify the contents of the package without having to re-run the pip install command



Dependency Management: pip vs conda

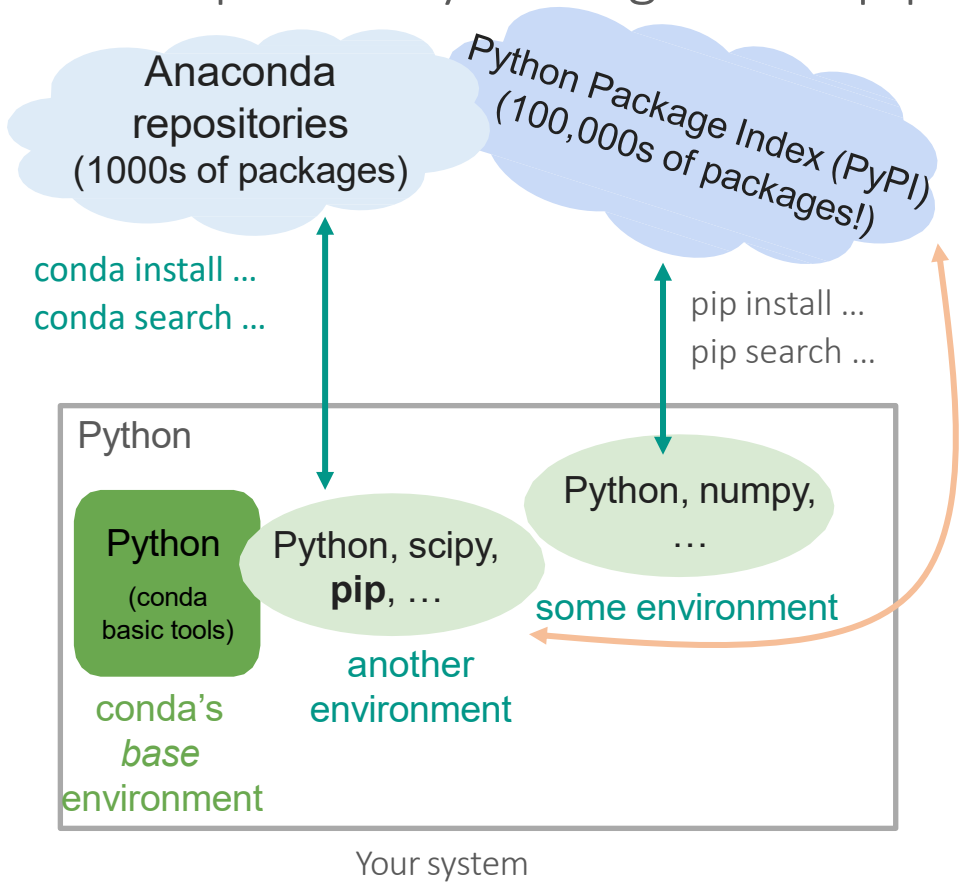


Two shortcomings of pip:

- To keep your system's Python packages intact, you need other software to create isolated environments
- Dependency tracking across packages is not the best

3rd-party software/package (virtualenv or venv are popular, but many options exist - it's a bit messy)

Dependency Management: pip vs conda



Conda lets you create isolated environments, manage packages *and* install PyPI packages. A typical workflow:

1. Create an environment (ex: a python 3.8 env) and activate it

$ conda create --name race python=3.8

$ conda activate race

2. Install all dependencies available in Anaconda's repos

$ conda install -c anaconda numpy

$ conda install -c conda-forge matplotlib

3. Install other dependencies only available through pip, if any

$ pip install …

4. Install your local package

$ cd lsy_drone_racing

$ pip install -e .

Useful: conda cheat sheet

Dependency Management: pip vs conda

Frequent use case: Sharing conda environments across robots, PCs, remote servers, OSes

Solution: *environment.yml* file

How this environment file is created will affect cross-platform compatibility and reproducibility:

Option 1: dump the list of conda and pip packages (including dependencies). Very good for reproducibility on similar OSes, bad for cross-platform compatibility

Dump all the packages: $ conda env export > environment.yml

On the other machine: $ conda env create -f environment.yml

Option 2: manually write the environment.yml file and *only* include packages installed through the command line. Good for cross-platform compatibility, ok-ish for reproducibility.

Dependency Management: pip vs conda

- Conda is a good starting point if you desire a simple setup
  - Install it using Miniconda (the full Anaconda installer is way too large)
  - A useful conda replacement is mamba (C++ implementation of conda - it's much faster)
  - Pro tip: Micromamba. No, this is not a joke
  - Use pip inside your conda environment only if needed

- Go the extra mile and manually write your environment.yml files

- Useful sources:
  - *Conda vs Pip: Choosing your Python package manager*
  - *Reproducible and upgradable Conda environments with conda-lock*

Software Development

Dependency Management: uv

- uv is a pip replacement written in Rust
- Can also handle virtual environments
- Significantly faster

- Cons:
- No support for system packages (e.g. OpenMPI, cuda, compilers etc.)

Software Development

Dependency Management: Pixi

- Built on **uv** for fast Python package resolution while handling **system-level libraries**
- Ensures a **non-leaking** environment

Capability	Pixi	Docker	Poetry/Uv	Conda/Mamba	Pip	Nix
Reproducibility	✓✓✓	✓✓	✓✓	×	×	✓✓✓
Native lockfiles	*Yes*	*Container isolation*	*Python only*	*Manual pinning*	*Manual pinning*	*Functional builds*
Multi-language Support	✓✓✓	✓✓	✓	✓✓	✓	✓✓✓
C++, Python, Rust, etc.	*All languages*	*Via containers*	*Python + C extensions*	*Most languages*	*Python + C extensions*	*All languages*
Hardware Integration	✓✓✓	✓	✓✓✓	✓✓✓	✓✓✓	✓✓
CUDA, Apple Silicon, etc.	*Native access*	*Basic support*	*Native access*	*Native access*	*Native access*	*Slow CUDA adoption*
Setup Complexity	✓✓✓	×	✓✓	✓	✓✓	✓✓
Single command install	*One command*	*Dockerfile creation*	*Python projects*	*Environment setup*	*Simple install*	*Simple with flakes*
Cross-platform	✓✓✓	✓	✓✓✓	✓✓✓	✓✓✓	✓
Linux, macOS, Windows	*All platforms*	*Platform dependent*	*All platforms*	*All platforms*	*All platforms*	*No Windows*
Learning Curve	✓✓✓	×	✓✓✓	✓✓	✓✓✓	×
Researcher accessibility	*Minimal*	*High (containers)*	*Minimal*	*Moderate*	*Minimal*	*Complex language*
System Dependencies	✓✓✓	✓✓	×	✓✓✓	×	✓✓✓
Compilers, libraries, etc.	*Fully managed*	*Via base images*	*Not handled*	*Fully managed*	*Not handled*	*OS-level management*

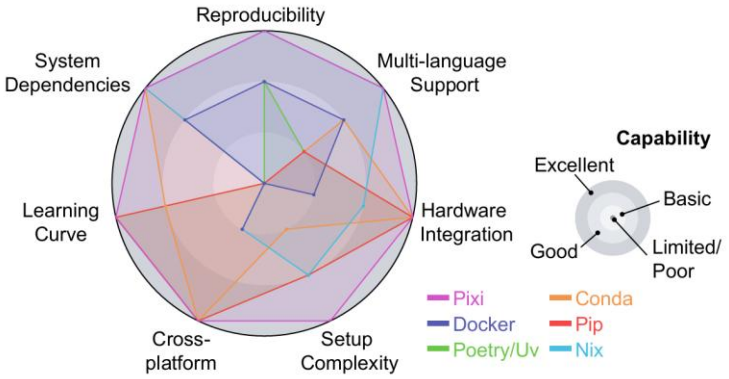


Table & Figure are sourced from: Pixi: Unified Software Development and Distribution for Robotics and AI

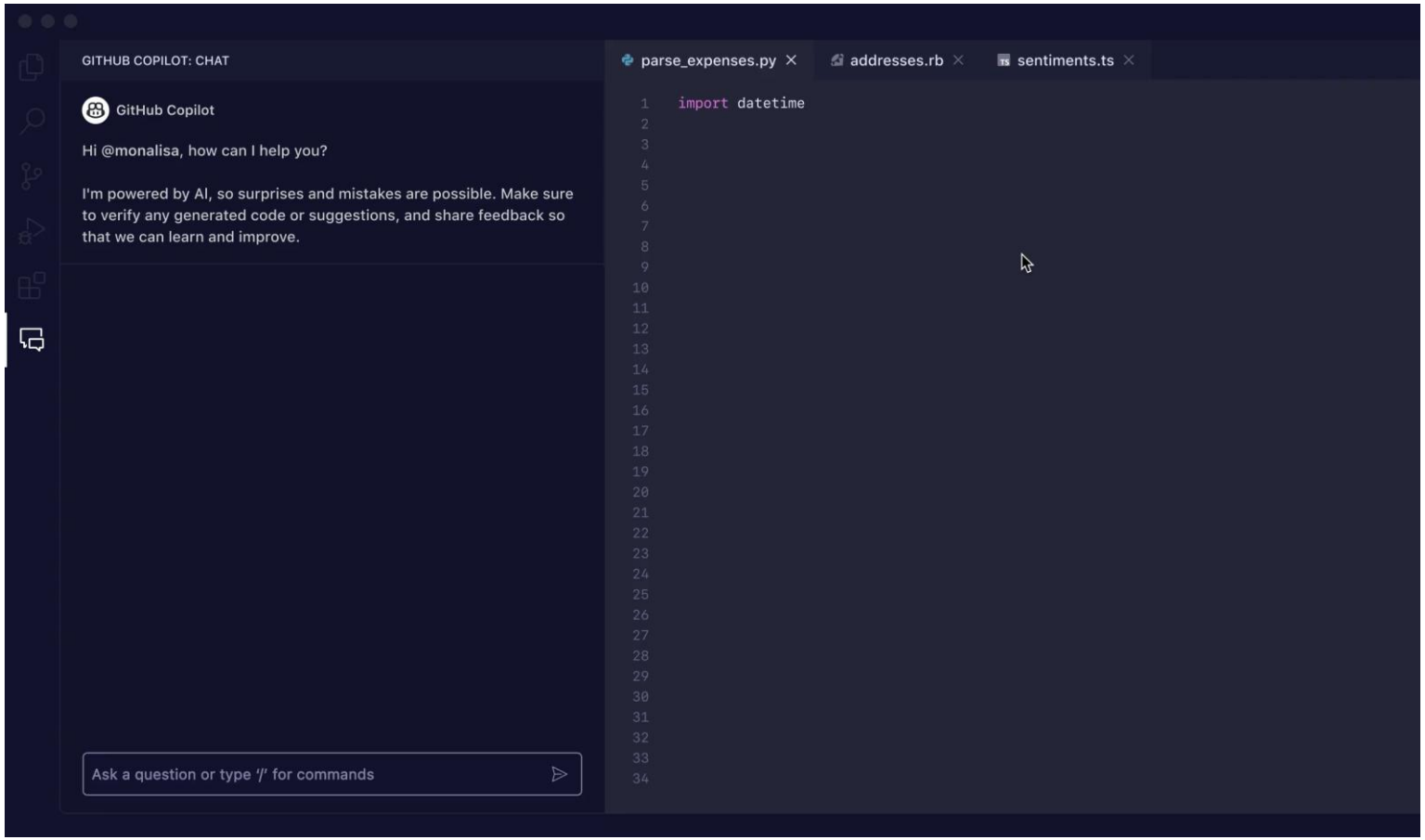
Core Commands: Pixi

- **`pixi init`**: Creates a new project with a pixi.toml configuration file.
- **`pixi add <package>`**: Installs a package and updates your lockfile.
- **`pixi add --pypi <package>`**: Specifically adds a Python package from PyPI.
- **`pixi install`**: Installs the project's virtual environment.
- **`pixi shell`**: Installs & activates the project's default environment in the current terminal.
- **`pixi shell -e <env_name>`**: Installs & activates a specific environment (e.g., sim or deploy) if your project has multiple configurations.
- **`exit (Ctrl+D)`**: Safely leaves the Pixi shell and returns to your system's standard terminal.
- **`pixi run <command>`**: Executes a command or a predefined task directly.

Other Remarks: Advanced Tools

GitHub Copilot

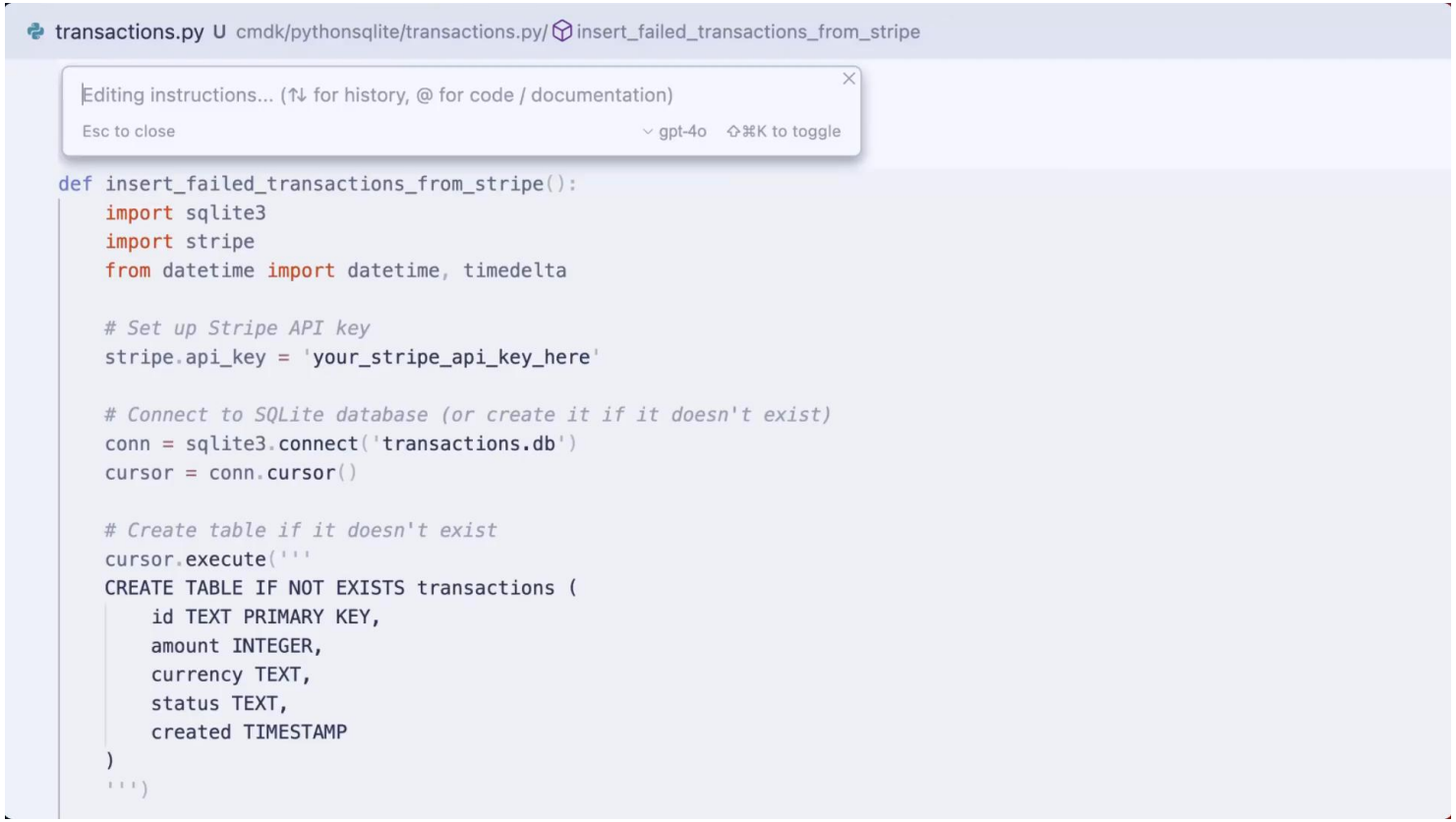
- Tool for supporting scripting process
- Generally, make things faster but require careful checking
- Free for students and instructors (see Quickstart Guide for GitHub Copilot)



Other Remarks: Advanced Tools

Cursor

- VSCode fork focused on AI integration
- Easy code generation and chatting with coding-specialized LLMs
- Free version available
- You can bring your own API key



Other Remarks: Debugging Python

- Pair programming: Coding in pairs with one driver and one observer and switching roles from time to time
  - Dedicated roles (one focuses on implementation, the other focuses on higher-level review and brainstorming ideas for improvements)
  - Minimize chances of getting errors
  - Transfer knowledge between programmers
- Rubber duck debugging: Explaining the code to a “rubber duck”
  - Idea is to debug by explaining the code or teaching it to an inanimate object
  - A process that forces going through the thought process on a deeper level

Additional Resources

- ETH ROS Course
  https://rsl.ethz.ch/education-students/lectures/ros.html
- The Robotics Toolbox by Prof. Peter Corke in Matlab and Python
  https://petercorke.com/toolboxes/robotics-toolbox/
- Industry Best Practices in Robotics Software Engineering
  https://arxiv.org/abs/2212.04877
- Python Code Collection for Robotics
  https://arxiv.org/pdf/1808.10703

Additional Resources

- Summer schools
  - OSIP 2017 https://archiveweb.epfl.ch/osip2017.epfl.ch/
  - RCS 2019 http://rcs18.ethz.ch/
  - OSIP 2019 https://archiveweb.epfl.ch/osip2019.epfl.ch/
- Gael Varoquaux (general advice, model-view-control) https://www.slideshare.net/GaelVaroquaux/computational-practices-for-reproducible-science
- Wenzel Jakob (general advice, cost-benefit analysis) https://drive.google.com/file/d/1yuna9RLU8yTD_ye1XnYbvNjhSwYMpl29/view
- Tim Head (binder, sharing jupyter notebooks) https://hackmd.io/@jVeyrAXKRpWC3e6Q9Evo6g/binder-osip2019#/3

Additional Resources

- General environment structure
  - The Hitchhiker's Guide to Python: Structuring Your Project
  - How to organize your Python data science project
- Naming conventions
  - The most important: stick to a convention!
  - Most commonly accepted: PEP 8 – Style Guide for Python Code
  - Google Python Style Guide
- setuptools (pyproject.toml, setup.py, setup.cfg)
  - A Practical Guide to Setuptools and Pyproject.toml
  - Configuring setuptools using pyproject.toml files

What We Cover Today

1. Software Development* [45 min]
2. Hardware Intro [45 min]
3. Time for Questions [25 min]

*Part of the content was adopted from the tutorial, “Code repository setup: lessons learned & getting hands-on with a Python repository,” by Frederike Dumbgen and Olivier Lamarre

Timeline

# Week	Dates	Content	Deadlines	Notes
1	15/10/2025	Lec: Kickoff		w/ Angela
2	22/10/2025	Lec: Project intro and setup		w/ Marcel
3	29/10/2025	Lec: Selected algorithms for racing		w/ Marcel
4	05/11/2025	Office Hour	05/11/2025 Submit to online challenge	w/ Yufei & Jiaming
5	12/11/2025	Lec: Hardware, code, tools + Lab tour		w/ Niklas
6	19/11/2025	*Lab sessions*		
7	26/11/2025	*Lab sessions*		
8	03/12/2025	Progress presentations	02/12/2025 Submit slides	
9	10/12/2025	*Lab sessions*		
10	17/12/2025	*Lab sessions*		
11	24/12/2025	Christmas break		
12	31/12/2025	Christmas break		
13	07/01/2026	*Lab sessions*		
14	14/01/2026	*Lab sessions*		
15	21/01/2026	Final presentation, Demo session on 23/01/2026 with open lab day	20/01/2026 Submit report, slides, code	w/ Angela

Progress Presentation: Guidelines

- The Progress Presentation **should include** the following elements:
  - **1. Methodology / Approach (60%)**
    Explain the reasoning behind your approach: Use clear figures (e.g., block diagrams, illustrations) to summarize the core idea and show how it addresses the project challenges/ Why this method and not another?/ Key advantages & limitations
  - **2. Progress to Date (20%)**
    Describe what you have achieved so far: What is working already/ What didn't work as expected/ Technical challenges you encountered along the way/ How and if you solved these challenges
  - **3. Project Milestones and SMART Goals (20%)**
    Outline the plan for the rest of the semester: Specific milestones and SMART goals/ What should be finished when/ Who is responsible for each task (be explicit about task assignments)
- **Do not include** Information that is identical for all groups (e.g., the assignment description, race track details, the hardware, …)
- The presentation should be **5 minutes long** and **all members should equally contribute to the presentation**. All members are required to attend this session. In case you need to present virtually, please notify us prior to the session.
- You should use this slide deck as a template for your presentation. **Submission deadline on Moodle is the day before the presentations, 23:59 CET.**

Lab Intro - Drone

Crazyflie Brushless

- Batteries
- Marker deck
- Replacement parts

Lab Intro - Environment

- XYZ axis
- Gates
- Mats
- Setup & Cleanup

Lab Intro – Motion Capture System

- How to activate drone & gates
- Temperature

Lab Intro – Deployment

- Lab PC or your PC
- Create new user account
- Clone your fork & setup
- Antennas
- Drone ID & Channel

Lab Intro – Safe Deployment Rules

- Stand behind the net, or wear safety googles
- Make sure nobody else is inside the area
- Always have on person with their hands on E-stop
- Test your implementation first in simulation
- First flight of your session should always be with an example controller to test hardware & setup
- There is a setting to fly without gates – use this before testing with gates!
- Start slow and get iteratively faster

Lets try to prevent breaking the drones as much as possible – takes time to fix, and costs money

Lab Intro – Debugging Deployment

- Check Drone ID & Channel
- Ping vicon PC/ rviz visualization
- Test with example controller
- Switch Radio/ Drone/ Restart PC
- Switch Battery

How to Safely Work With Robots

- Think of all things that could possibly go wrong.
- Have a safe countermeasure for each option that minimizes damage with priority on minimizing personal injuries and, second, minimizing equipment damage.

Example:

- Crazyflie moves uncontrollably
- Battery explodes
- Lighthouse base station on tripod falls on someone’s head
- Someone walks into the flight space

How to Safely Work With Robots

Countermeasures

- Crazyflie moves uncontrollably
  - Wear safety glasses or place yourself behind the net
  - Have a software emergency land/switch-off
  - Protect yourself behind glass wall or by leaving the room and wait till battery is empty
- Battery explodes
  - Put it in safety bag
  - Call fire department if serious
  - Inform all lab members
- Lighthouse base station on tripod falls on someone's head
  - Turn off all robots
  - Check if you need to call an ambulance
- Someone walks into the flight space
  - Choose the safest option to stop the robots (e.g., call the person out, continue your program and land afterwards, or immediately press the emergency stop)

How to Safely Work With Robots

When do accidents happen most often?

- During the initial setup phase where you do not know the equipment well
- When being in a rush (e.g., when turning off the system to catch your train)
- When becoming very comfortable with the system (e.g., when running the 40th experiment of the day)
- When doing things for a special occasion (e.g., going inside the protected area for filming)

How to Safely Work With Robots

- Think of all things that could possibly go wrong.
- Have a safe countermeasure for each option that minimizes damage with priority on minimizing personal injuries and, second, minimizing equipment damage.
- Define and develop “safety layers”
  - Physical protection of the environment (e.g., nets)
  - Various software safety layers (e.g., emergency robot power off, emergency landing, stop of your program and robots hovering)
  - Last resort: personal physical protection (e.g., wearing gloves and safety glasses)

Example: https://robotics.illinois.edu/robot-manipulator-safety-rules/

What We Cover Today

1. Software Development* [45 min]
2. Hardware Intro [45 min]
3. Time for Questions [25 min]

*Part of the content was adopted from the tutorial, “Code repository setup: lessons learned & getting hands-on with a Python repository,” by Frederike Dumbgen and Olivier Lamarre